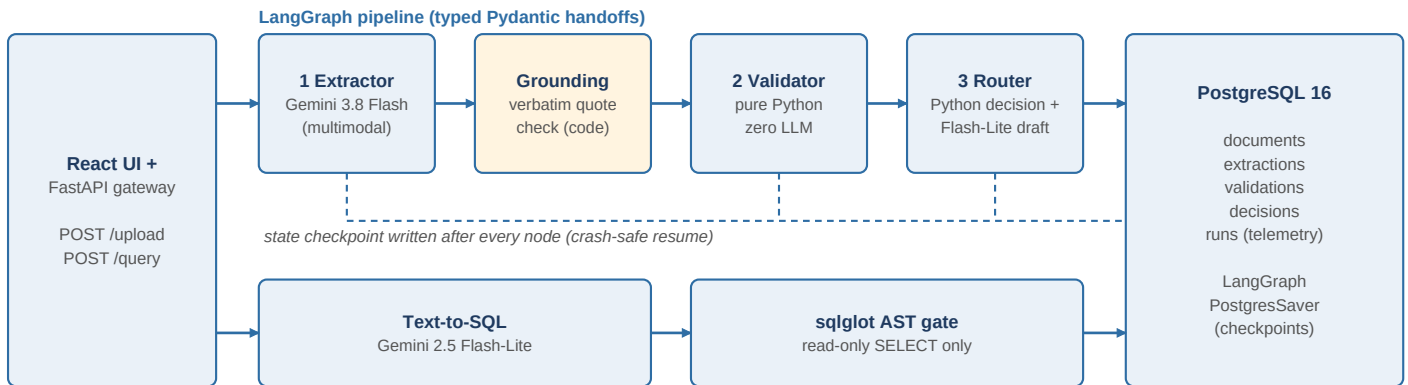


Nova DAW: Technical Write-up (Part 1)

Autonomous trade document verification pipeline | v1.0.0 | Author: Akash Jagtap | Guarantee: **zero silent approvals** (0.0% false-approve rate on the discrepant / uncertain golden documents)

1. Architecture: where data flows and where state lives



Perception, rule validation and policy routing are separate nodes in one LangGraph graph. The **Extractor** reads the document with a multimodal LLM and returns fields with confidence and a verbatim source quote. A code-level **grounding** step confirms each quote exists in the raw text. The **Validator** is pure Python (regex, token-sort Levenshtein, bounds checks) and has **zero LLM calls**, so a model cannot hallucinate rule compliance. The **Router** picks approve / review / amend in Python *before* the LLM drafts the explanation or supplier email. State lives in PostgreSQL: business tables plus LangGraph checkpoints of `TradePipelineState`. After a SIGKILL or DB interruption the job resumes idempotently from the last checkpoint, with no upstream re-extraction and no duplicate LLM cost.

Why three agents. One prompt suffers "hallucination of conformity": asked to transcribe and verify, the model "corrects" discrepancies (fixes digits, assumes an Incoterm). Splitting into 5+ per-field agents multiplies round-trips, inflates token overhead by about 400% and pushes latency from ~1.2 s to over 6.5 s per document.

Role	Model	Price / 1M tokens (in / out)	Rationale
Extractor	gemini-3.8-flash	\$0.15 / \$0.60	Multimodal OCR, native schema-enforced output, fast time to first token (~450 ms)
Extractor fallback	gemini-3.1-pro-preview	\$1.25 / \$5.00	Automatic escalation when confidence is below 0.85 or text density is low (messy scans)
Validator	None (pure Python)	\$0 / \$0	Deterministic rules: no tokens, no drift
Router and Text-to-SQL	gemini-2.5-flash-lite	\$0.075 / \$0.30	Cheap drafting and SQL generation; decision is gated by Python, SQL by the AST check

2. Three nastiest failure modes, from my own test runs

Failure mode	Real case (12-doc golden set)	What went wrong and how it is stopped
1. HS code digit transposition	doc_04: declared 8479.05.00; customer-approved code is 8479.50.00	Problem: in early prototypes with unconstrained extraction, the LLM often "normalized" the code to the approved one, assuming an OCR typo. The shipment would have cleared under a wrong tariff code (customs impoundment, port fines). Fix: the exact string is extracted with its verbatim quote; the Validator compares 6-digit prefixes (847950 vs 847905), returns MISMATCH ("HS code 8479.05.00 is not in customer approved list") and routes to amendment_request.
2. Consignee legal-suffix drift	doc_09: "Meridian Robotics LLC" (approved alias) passed at 100%. doc_05: "Acme Global Industrial Logistics Ltd" failed at 0.12 similarity	Problem: exact matching rejects valid suffix variants; loose substring matching lets an unauthorized forwarder claim cargo. Fix: two-tier matching. Exact match on name or alias = MATCH; token-sort similarity 0.85 or higher = MATCH; 0.65 to 0.84 = UNCERTAIN (forced human review, never auto-approve); below 0.65 = MISMATCH (amendment request).
3. Missing field filled by a model prior	doc_06: Incoterm omitted from the document entirely	Problem: multimodal LLMs have a strong prior from trade contracts and tend to invent "FOB" or "CIF" from port context, silently approving a contract with no terms of carriage. Fix: prompt demands null / 0.0 for absent fields; grounding asserts the quote is in the raw text, else is_grounding=False and confidence collapses to 0.0; the Validator forces ungrounded fields to UNCERTAIN, so the document goes to human_review.

3. Observability: tracing one shipment for 50 customers

Every LLM call, parser step and validation event writes a row to the `runs` table: `document_id`, `node_name`, `model_name`, `latency_ms`, `prompt / completion / thinking tokens`, `cost_usd`, `status` and `error_message`. `document_id` joins a shipment's runs to its extraction, validation and decision rows, so one query reconstructs the full path from input to verified output. Today the UI shows header micro-stats (total runs, aggregate cost, average latency) and a telemetry drawer with per-node traces. Email ingestion is not built yet (an IMAP / webhook adapter would feed `/api/documents/upload`), so email-to-output tracing starts at upload.

Production dashboard and alerts (defined thresholds):

- Thinking-token runaway:** more than 4,000 thinking tokens flags and throttles the run.
- Node latency:** Extractor or Router above 5,000 ms alerts operations and adjusts retry backoff.
- Fallback rate:** more than 10% of 100 documents needing vision OCR or Pro escalation triggers a scanner / ingestion quality check.
- Next:** OpenTelemetry spans around each node (`extractor_span`, `validator_span`, `router_span`) with traceparent propagation from UI to backend, or Langfuse, for multi-cluster deployments.

4. Cost per document

Stage	Model / component	Prompt tok	Completion tok	Thinking tok	Cost (USD)
Extractor	gemini-3.8-flash	850	180	120	\$0.000251
Grounding, Validator, Storage	In-memory / rules / Postgres	0	0	0	\$0.000000
Router	gemini-2.5-flash-lite	320	110	45	\$0.000065
Total (clean pass)	Full pipeline	1,170	290	165	\$0.000316

Per 1,000 documents: **\$0.32** clean, **\$0.48** when amendment drafting is needed, **\$0.85** when messy scans need the Pro fallback (at a 5% rate). **Where it blows up:** thinking tokens on ambiguous documents and the Pro fallback, which costs roughly 8 times the Flash rates per token. **Controls:** deterministic Validator and decision (no tokens), Flash-Lite for drafting, per-call cost logged to `runs`, the 4,000 thinking-token throttle and the 10% fallback-rate alert.

5. Latency

Execution profile	P50	P90	P95	Max
Text-layer PDF (clean)	72 ms	104 ms	126 ms	148 ms
Synthetic eval suite (12 docs)	76 ms	102 ms	118 ms	134 ms
End-to-end, live Gemini calls	1,180 ms	1,640 ms	1,950 ms	2,420 ms
Text-to-SQL query endpoint	420 ms	680 ms	820 ms	980 ms

Measured on a local workstation with Docker Compose. The gap between ~75 ms and ~1.2 s shows the **LLM calls (mainly the multimodal Extractor) are the slowest hop**; everything else is in-process code. Levers already in place: the fast Flash tier on the default path, Pro only on the ~5% fallback, only two LLM hops (per-field agents would take latency past 6.5 s), and the 5,000 ms per-node alert. With more time, OpenTelemetry spans would show exactly which node drives the tail.

6. What I would do differently with a week instead of a day

- **3-document cross-validation.** Extend state from one `ExtractedDoc` to a bundle (`bol`, `commercial_invoice`, `packing_list`) with a dedicated cross-referencing validator node.
- **Human-in-the-loop approval.** Use LangGraph's native `interrupt()` to pause at `HUMAN_REVIEW` until an operator approves or amends in the UI, before any supplier email goes out.
- **Automated inbound ingestion.** Add an IMAP / webhook adapter in front of the existing upload endpoint, with no change to agent code.
- **Production observability.** Move from the native `runs` table to OpenTelemetry or Langfuse traces for multi-cluster deployments.

What worked best: the grounding invariant removed every digit hallucination and phantom Incoterm on the golden set; zero-LLM validation kept the model from "forgiving" compliance errors; and the sqlglot gate blocked `DROP`, `UPDATE`, `INSERT` and multi-statement queries.